CAMPOS IA
APOSTILA TÉCNICA

Ferramentas de Programação

Curso Técnico em Informática / Tecnologia

Público-alvo: Alunos de Curso Técnico

Carga horária sugerida: 20 horas

Nível: Básico a Intermediário

Versão: 1.0 — 2026

Material didático exclusivo da plataforma CAMPOS IA

www.camposia.com.br • Uso interno para alunos

Sumário

1. Introdução à Programação 3
2. Linguagens de Programação 4
3. Compiladores e Interpretadores 5
4. IDEs e Editores de Código 6
5. Controle de Versão com Git 7
6. Debugging e Boas Práticas 8
7. Ferramentas Modernas 9
8. Exercícios de Fixação 10

Objetivo desta apostila: Apresentar as principais ferramentas e conceitos usados no desenvolvimento de software, preparando o aluno para escrever, organizar, versionar e depurar código de forma profissional.

1. Introdução à Programação

1.1 O que é Programar?

Programar é o ato de escrever instruções (código) que um computador consegue executar. Essas instruções resolvem problemas, automatizam tarefas, processam dados ou criam aplicações interativas. A programação é a base de praticamente tudo que envolve software: sites, aplicativos, sistemas empresariais, jogos e soluções de Inteligência Artificial.

1.2 Por que aprender ferramentas?

Saber a sintaxe de uma linguagem é apenas o começo. Profissionais de TI usam um conjunto de ferramentas para:

- Escrever código de forma produtiva (editores e IDEs)
- Organizar e colaborar em projetos (Git e repositórios)
- Encontrar e corrigir erros (debuggers e logs)
- Testar e entregar software de forma confiável (CI/CD, containers)
- Reutilizar código e bibliotecas (gerenciadores de pacotes)

1.3 Ambiente de Desenvolvimento

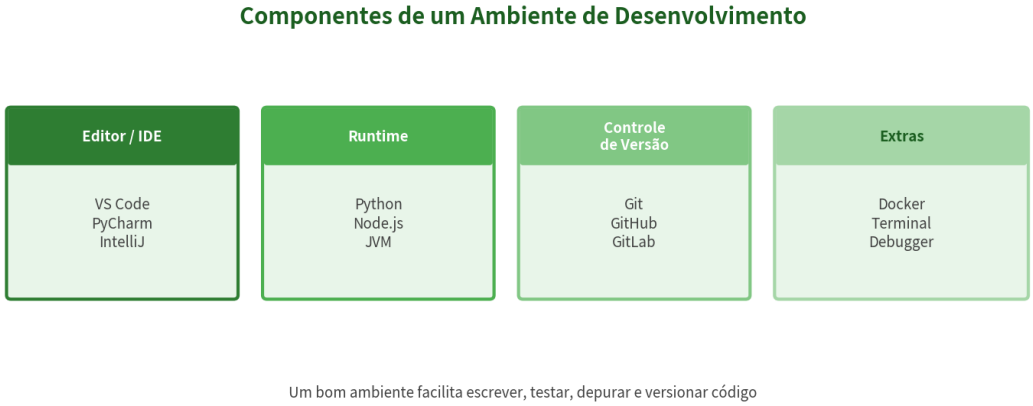


Figura 1 — Componentes típicos de um ambiente de desenvolvimento

Dica: Comece simples: instale o VS Code + Python (ou Node.js) + Git. Com essas três ferramentas você já consegue praticar a maioria dos exercícios de um curso técnico.

2. Linguagens de Programação

Existem centenas de linguagens. Elas se diferenciam pelo nível de abstração, paradigma (orientada a objetos, funcional, etc.) e propósito (web, dados, sistemas, mobile).

2.1 Níveis de Linguagem



Figura 2 — Classificação por nível de abstração

2.2 Linguagens Mais Usadas no Mercado Técnico

Linguagem	Tipo	Uso principal	Dificuldade inicial
Python	Interpretada	Dados, automação, IA, web	Baixa
JavaScript	Interpretada	Web (front e back)	Baixa-Média
Java	Compilada*	Enterprise, Android	Média
C / C++	Compilada	Sistemas, embarcados, jogos	Alta
SQL	Declarativa	Bancos de dados	Baixa

** Java compila para bytecode e roda na JVM.*

3. Compiladores e Interpretadores

Para que o código que escrevemos seja executado pelo processador, ele precisa ser traduzido. Existem duas abordagens principais:

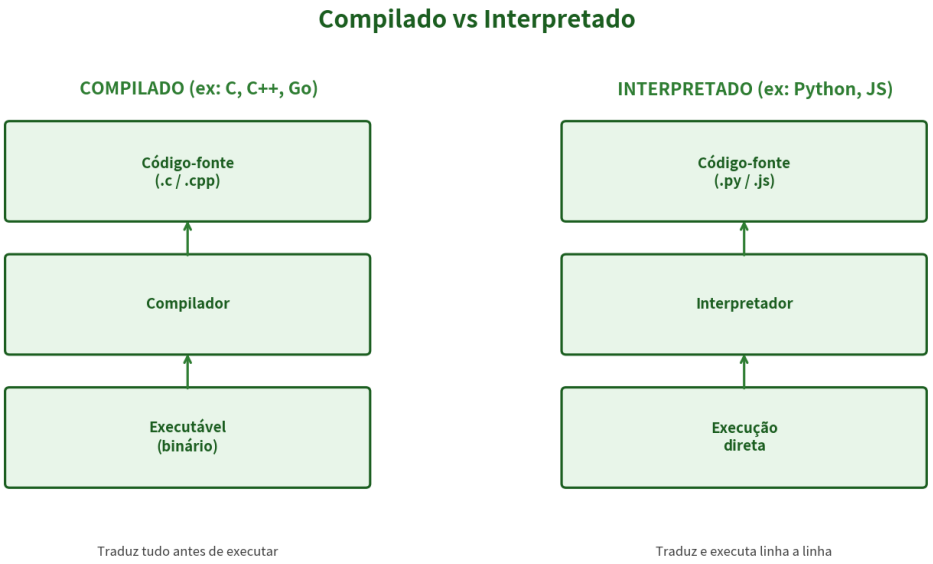


Figura 3 — Fluxo compilado versus interpretado

3.1 Comparativo

Aspecto	Compilado	Interpretado
Tradução	Todo o código antes de executar	Linha a linha (ou sob demanda)
Velocidade de execução	Geralmente mais rápida	Geralmente mais lenta
Portabilidade	Precisa recompilar por plataforma	Roda onde houver interpretador
Exemplos	C, C++, Go, Rust	Python, JavaScript, Ruby

Existem também abordagens híbridas (como Java e C#), que compilam para um código intermediário (bytecode) executado por uma máquina virtual.

4. IDEs e Editores de Código

4.1 Editor vs IDE

• Editor de código: ferramenta leve focada em editar texto (ex.: VS Code, Sublime Text, Neovim).
• IDE (Integrated Development Environment): ambiente completo com editor, debugger, terminal, gerenciador de projetos e integração com ferramentas (ex.: PyCharm, Visual Studio, IntelliJ IDEA).

4.2 Visual Studio Code (recomendado para o curso)

O VS Code é atualmente o editor mais popular do mundo. É gratuito, multiplataforma e extremamente extensível por meio de extensões. Recursos importantes:

- Destaque de sintaxe e autocompletar inteligente (IntelliSense)
- Terminal integrado
- Debugger visual
- Integração nativa com Git
- Milhares de extensões (Python, JavaScript, Docker, etc.)
- Suporte a Jupyter Notebooks e desenvolvimento remoto

4.3 Outras opções relevantes

Ferramenta	Foco	Observação
PyCharm	Python	Excelente para projetos Python profissionais
IntelliJ IDEA	Java / Kotlin	Padrão de mercado para Java
Visual Studio	.NET / C++	IDE completa da Microsoft (Windows)
Android Studio	Android	Baseada no IntelliJ, oficial do Google

5. Controle de Versão com Git

O Git é o sistema de controle de versão mais usado no mundo. Ele permite registrar o histórico de alterações do código, trabalhar em equipe sem sobrescrever o trabalho dos colegas e recuperar versões anteriores quando necessário.

5.1 Fluxo Básico

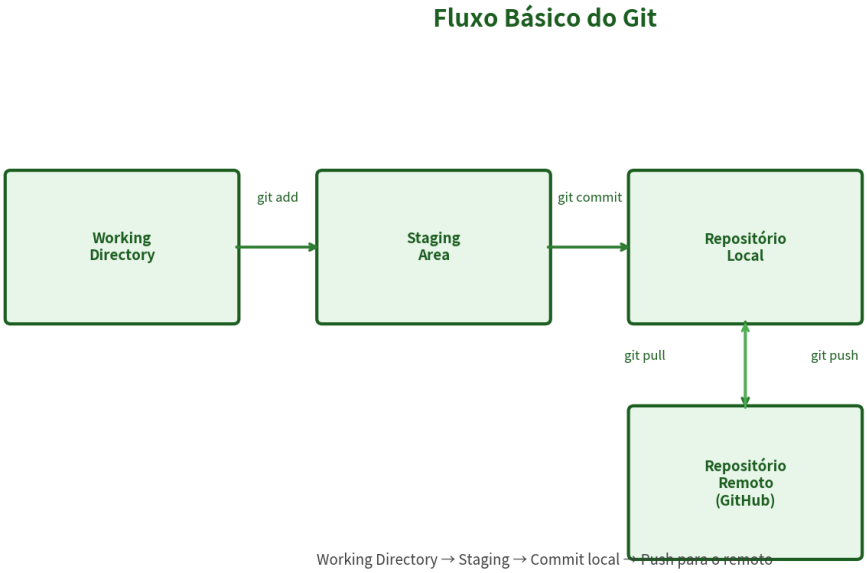


Figura 4 — Fluxo básico: Working Directory → Staging → Commit → Remoto

5.2 Comandos Essenciais

Comando	Função
git init	Inicia um repositório local
git clone	Copia um repositório remoto
git status	Mostra o estado atual dos arquivos
git add .	Adiciona alterações à Staging Area
git commit -m "msg"	Registra um commit com mensagem
git push	Envia commits para o remoto
git pull	Baixa e integra alterações do remoto
git log	Exibe o histórico de commits
git branch / checkout	Cria e troca de branches

Plataformas: GitHub, GitLab e Bitbucket hospedam repositórios remotos e oferecem recursos de colaboração (Pull Requests, Issues, Actions).

6. Debugging e Boas Práticas

6.1 O que é Debugar?

Debugar (ou depurar) é o processo de encontrar e corrigir erros (bugs) no código. É uma das habilidades mais importantes de qualquer programador.

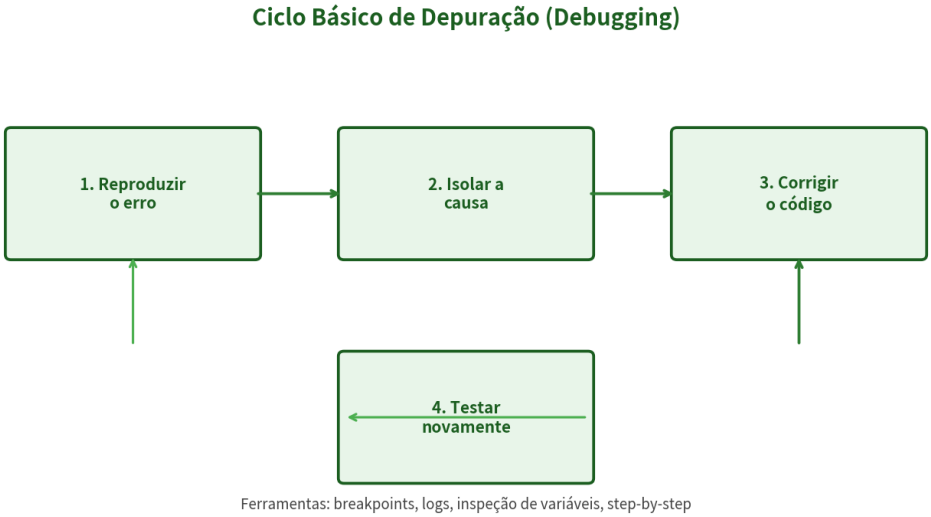


Figura 5 — Ciclo básico de depuração

6.2 Técnicas Comuns

- Print / log: inserir mensagens para ver valores de variáveis e fluxo de execução.
- Breakpoints: pausar a execução em uma linha específica e inspecionar o estado.
- Step-by-step: executar o código linha a linha (step over, step into).
- Leitura de mensagens de erro: stack traces indicam onde e por que o programa falhou.

6.3 Boas Práticas Iniciais

- Use nomes de variáveis e funções claros e significativos.
- Escreva código legível (indentação, espaços, comentários quando necessário).
- Faça commits pequenos e com mensagens descritivas.
- Teste seu código com frequência (não espere juntar tudo no final).
- Leia a documentação oficial das ferramentas e linguagens.
- Peça e ofereça revisão de código (code review) sempre que possível.

7. Ferramentas Modernas

Além do editor e do Git, o ecossistema atual oferece ferramentas que aumentam muito a produtividade e a qualidade do software.

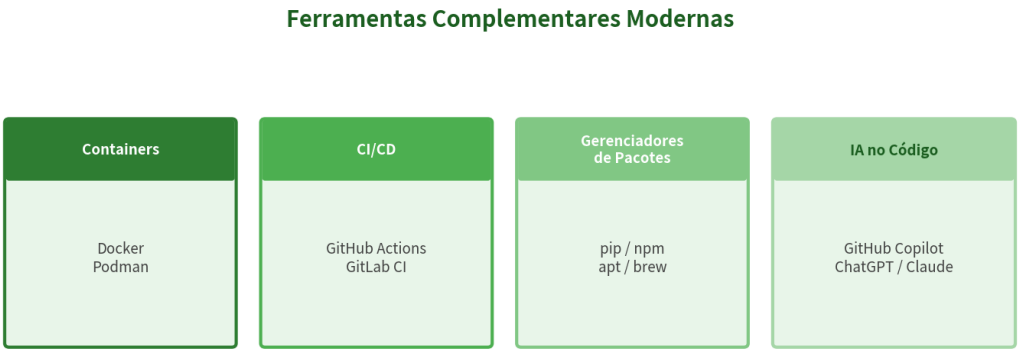


Figura 6 — Ferramentas complementares modernas

7.1 Resumo das Categorias

• Containers (Docker): empacotam a aplicação e suas dependências, garantindo que rode igual em qualquer ambiente.

• CI/CD: automação de testes e deploy (GitHub Actions, GitLab CI). Toda mudança no código pode ser testada automaticamente.

• Gerenciadores de pacotes: pip (Python), npm (JavaScript), apt/brew (sistema). Facilitam instalar e atualizar bibliotecas.

• IA no desenvolvimento: ferramentas como GitHub Copilot e ChatGPT ajudam a gerar código, explicar trechos e sugerir correções. Use com senso crítico.

Conselho final: Não tente aprender todas as ferramentas de uma vez. Domine bem o básico (editor + linguagem + Git + debugger) e vá incorporando as demais conforme a necessidade dos projetos.

8. Exercícios de Fixação

Responda às questões abaixo para consolidar o aprendizado.

1. Explique a diferença entre uma linguagem compilada e uma linguagem interpretada. Dê um exemplo de cada.

2. O que é uma IDE? Cite duas vantagens de usar uma IDE em relação a um editor de texto simples.

3. Descreva o fluxo básico do Git: o que acontece em cada etapa (Working Directory → Staging → Commit → Remoto).

4. Escreva a sequência de comandos Git para: clonar um repositório, criar um arquivo, adicioná-lo, fazer commit e enviar para o remoto.

5. O que são breakpoints e por que eles são úteis no processo de debugging?

6. Liste três boas práticas de programação que você pretende adotar a partir de agora.

7. Para que serve o Docker em um ambiente de desenvolvimento? Qual problema ele resolve?

8. Instale o VS Code (se ainda não tiver) e configure a extensão da linguagem que você está estudando. Faça um “Hello, World!” e tire um print da tela.

Gabarito e materiais complementares

O gabarito comentado, links de instalação e tutoriais práticos estão disponíveis na área do aluno da plataforma CAMPOS IA.

CAMPOS IA — Formação técnica de qualidade
Esta apostila conclui o conjunto básico de materiais do módulo técnico.
Parabéns por chegar até aqui!